

NOMBRES: YURI ANTONIO FUENTES ZAMBRANO

CARRERA: TECNOLOGIA DE LA INFORMACION

PARALELO: “A”

SEMESTRE: “5to SEMESTRE”

MATERIA: DESARROLLO DE SISTEMAS INFORMATICOS

FACULTA: CIENCIAS INFORMATICAS

ACTIVIDA #1

TITULO: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

FECHA: 18/10/2026

DOCENTE: Ing. Jorge Arun Zambrano Cedeño

MAGISTER: Desarrollo de Sistemas Informáticos

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Estudio de caso: Sistema de Control de Rescate y Adopción Animal

El Sistema de Control de Rescate y Adopción Animal es un proyecto de software que busca facilitar el registro, seguimiento y administración de animales rescatados, así como gestionar sus procesos de atención veterinaria y adopción responsable.

1. Propósito del documento

Este documento explica el diseño del sistema de gestión de casos de un refugio animal. Describe qué patrones de diseño se aplicaron y qué problema concreto resuelve cada uno, por qué el equipo trabaja con Gitflow, y cómo se reflejan los patrones en los diagramas de clases y de secuencia.

El sistema permite reportar ingresos de perros, gatos y fauna silvestre, asignar voluntarios o veterinarios responsables y gestionar los estados del caso (por ejemplo, Rescatado, En tratamiento, Adopción o Liberación). (Pilatuña Aguagallo, 2023)

Objetivos

Objetivo general

Desarrollar un sistema informático para gestionar y controlar los procesos de rescate, atención y adopción de animales, facilitando el acceso a la información y mejorando la administración del refugio. (Carrión, 2021)

Objetivos específicos

- Registrar los animales rescatados con sus datos, fotografías y estado de salud.
- Administrar las consultas, vacunas y tratamientos veterinarios.
- Registrar las solicitudes de adopción y evaluar a los posibles adoptantes.
- Controlar las adopciones realizadas y los seguimientos posteriores.
- Generar reportes sobre rescates, animales disponibles y adopciones. (Gallegos Ortíz, 2018)

Usuarios del sistema

Usuarios	Funciones principales
Administrador	Gestionar usuarios, registros y reportes.
Rescatista	Registrar rescates y datos del animal.
Veterinario	Registrar diagnósticos, vacunas y tratamientos
Responsable de adopciones	Revisar solicitudes y registrar adopciones.
Adoptante	Consultar animales disponibles y presentar solicitudes.

Requerimientos funcionales

Los requerimientos funcionales describen las acciones que el sistema debe permitir.

1. Registrar nuevos animales rescatados.
2. Actualizar los datos y el estado de cada animal.
3. Registrar información médica y tratamientos.
4. Consultar los animales disponibles para adopción.
5. Registrar y evaluar solicitudes de adopción.
6. Aprobar o rechazar solicitudes.
7. Registrar la entrega del animal al adoptante.
8. Realizar seguimiento después de la adopción.
9. Generar reportes estadísticos.

Requerimientos no funcionales

- Seguridad: permitir el acceso según el rol del usuario.
- Usabilidad: presentar una interfaz sencilla y comprensible.
- Integridad: evitar registros incompletos o inconsistentes.
- Disponibilidad: facilitar el acceso al sistema cuando sea necesario.
- Rendimiento: permitir consultas rápidas de los registros.
- Privacidad: proteger los datos personales de los adoptantes.

2. Visión general de la arquitectura

Se eligió un monolito modular en capas, donde las dependencias apuntan siempre hacia el dominio. Para una fundación con pocos recursos es más sencillo de desplegar y mantener que una arquitectura de microservicios, y deja las fronteras listas por si algún módulo se separa en el futuro.

El Sistema de Gestión de Rescate y Adopción Animal tiene como finalidad organizar y automatizar los procesos relacionados con el rescate, registro, atención veterinaria y adopción responsable de animales. Para lograrlo, se propone una arquitectura lógica que separe las responsabilidades del sistema, facilite el mantenimiento del software y permita incorporar nuevas funcionalidades sin afectar los módulos existentes.

Propuesta de arquitectura lógica

Se propone una arquitectura por capas, complementada con módulos funcionales. Este enfoque es apropiado para un proyecto académico porque permite separar la interfaz, las reglas del negocio y el acceso a los datos.

El diseño se fundamenta en principios de programación orientada a objetos, patrones de diseño de software, separación de responsabilidades y buenas prácticas de control

de versiones mediante Gitflow. De esta manera, se busca construir una solución escalable, reutilizable, segura y mantenible.

Objetivo general

Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal mediante la aplicación de patrones de diseño de software y un flujo de trabajo profesional con Gitflow, promoviendo buenas prácticas de desarrollo y facilitando la evolución del sistema.

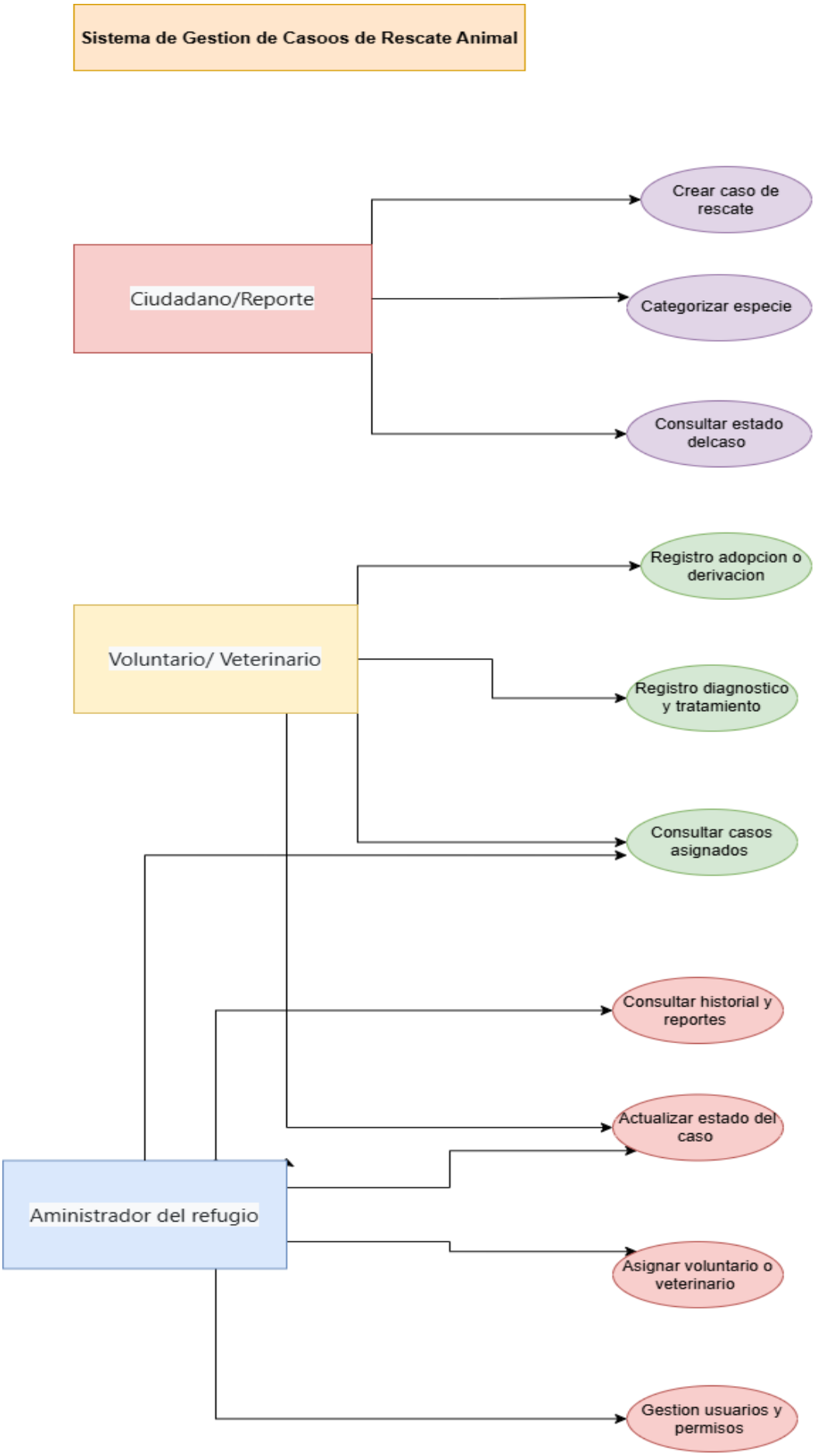
Objetivos específicos

- Identificar los módulos y las responsabilidades principales del sistema.
- Definir las capas de la arquitectura lógica y sus relaciones.
- Seleccionar patrones de diseño adecuados para resolver problemas específicos del sistema.
- Establecer un modelo de datos que permita gestionar animales, rescates, historiales médicos y adopciones.
- Definir un flujo de trabajo Gitflow para organizar el desarrollo colaborativo, las pruebas y las versiones del software.
- Promover la reutilización del código, la reducción del acoplamiento y la facilidad de mantenimiento.

Análisis del problema

En los refugios y organizaciones de protección animal, la información suele distribuirse entre registros manuales, hojas de cálculo y documentos independientes. Esta situación dificulta conocer el estado de cada animal, controlar su historial médico, administrar las solicitudes de adopción y realizar el seguimiento posterior.

Para resolver estos problemas, el sistema debe centralizar la información y organizar sus funciones en módulos independientes, con reglas de negocio claras y mecanismos que permitan ampliar el software en el futuro.


Sistema de Gestion de Casoos de Rescate Animal
Ciudadano/Reporte
Crear caso de rescate
Categorizar especie
Consultar estado delcaso
Voluntario/ Veterinario
Registro adopcion o derivacion
Registro diagnostico y tratamiento
Consultar casos asignados
Aministrador del refugio
Consultar historial y reportes
Actualizar estado del caso
Asignar voluntario o veterinario
Gestion usuarios y permisos

La organización interna sigue el patrón MVC:

Capa	Contenido	Responsabilidad
Modelo	Expedientes, fábricas, motor de casos, repositorio, observadores	Regla de negocio y acceso a datos
Vista	RescateVista	Como se muestran formularios y confirmaciones
Controlador	RescateController	Recibe el formulario, llama al modelo y elige la vista

Descripción de las capas

Capa de presentación

Es el medio por el cual los usuarios interactúan con el sistema. Contiene formularios, tablas, búsquedas, paneles y pantallas de administración. Envía las solicitudes a la capa de aplicación y muestra los resultados.

Capa de aplicación

Coordina los casos de uso, por ejemplo, registrar un rescate, actualizar el historial médico o aprobar una solicitud de adopción. Verifica los permisos y coordina las operaciones necesarias sin concentrar todas las reglas del negocio en la interfaz.

Capa de dominio

Contiene las entidades principales y las reglas que determinan el comportamiento del sistema. Por ejemplo, un animal no puede ser adoptado si no cumple las condiciones de adopción establecidas. Esta capa debe ser independiente de la interfaz y de la tecnología de base de datos.

Capa de infraestructura

Gestiona la persistencia de información, la implementación de los repositorios, los servicios de notificación y otras integraciones técnicas. Sus componentes concretos implementan las interfaces que necesita la aplicación o el dominio.

Flujo principal: un voluntario registra un rescate, el sistema abre el expediente correcto según la especie, lo guarda y avisa automáticamente a los voluntarios que pueden atenderlo.

3. Patrones de diseño aplicados

Se aplicaron cuatro patrones, uno más que el mínimo exigido. Ninguno se incluyó por adorno: cada uno responde a un problema real del refugio.

Singleton

Propósito: garantizar que exista una sola instancia de una clase y ofrecer un punto de acceso global a ella.

Problema en el sistema: si cada módulo crea su propia conexión a la base de datos o su propio motor de casos, aparecen varias conexiones abiertas y, peor aún, varios sujetos de eventos. Los observadores se suscribirían a uno y los rescates se publicarían en otro, así que las notificaciones a voluntarios nunca llegarían.

Solución: una metaclase SingletonMeta con doble verificación y candado (seguro para hilos). Se aplica a ConexionBD y a MotorCasos.

Beneficio: un solo punto de acceso a los datos y un solo bus de eventos en toda la aplicación.

Precauciones: los argumentos del constructor solo cuentan en la primera llamada, por eso la conexión se crea primero en main.py. Como el Singleton es estado global y complica las pruebas, se incluyó un método reiniciar () y se limitó su uso a dos clases donde la instancia única es un requisito real.

Factory Method

Propósito: delegar en subclases la decisión de qué objeto concreto crear.

Problema en el sistema: cada especie exige un expediente distinto. Perros y gatos llevan vacunas y esterilización, los gatos además una prueba de FeLV/FIV, y la fauna silvestre no se adopta: requiere aviso a la autoridad ambiental y valoración de un especialista. Resolverlo con if especie == ... dispersos obliga a tocar el motor cada vez que aparece una especie nueva.

Solución: la clase abstracta FabricaExpediente define el método fábrica crear_expediente () y un método plantilla abrir () que crea el expediente y lo valida antes de entregarlo. FabricaPerro, FabricaGato y FabricaSilvestre producen ExpedientePerro, ExpedienteGato y ExpedienteSilvestre.

Beneficio: para agregar una especie (ave, reptil) se crea un producto y una fábrica y se registran, sin modificar MotorCasos (principio abierto/cerrado). Además, ningún expediente nace sin pasar por su validación específica.

Observer

Propósito: definir una dependencia uno a muchos para que, cuando un objeto cambia, todos sus interesados reciban el aviso automáticamente.

Problema en el sistema: al registrar un rescate hay que avisar a los voluntarios, escribir la bitácora y, más adelante, tal vez actualizar estadísticas. Meter todo eso dentro de reportar_ingreso () acopla el motor con cada consecuencia posible.

Solución: SujetoRescates publica el evento RescateRegistrado y los observadores (NotificadorVoluntarios y RegistroBitacora) reaccionan por su cuenta. El notificador filtra por especie mediante un directorio de voluntarios, de modo que un rescate de

fauna silvestre solo llega a quienes están capacitados, y envía el mensaje a través de un CanalNotificacion intercambiable (consola, correo o WhatsApp). Si un observador falla, los demás siguen recibiendo el evento.

Beneficio: es la base reactiva del sistema. Agregar una reacción nueva (por ejemplo, generar un certificado) es añadir un observador, no editar el motor.

MVC (Modelo-Vista-Controlador)

Propósito: separar la lógica de negocio de la interfaz y de la coordinación entre ambas.

Problema en el sistema: voluntarios con poca experiencia técnica usarán formularios simples, y la interfaz debe poder cambiar (portal web, app móvil) sin reescribir las reglas del refugio.

Solución: RescateController recibe el formulario, invoca al modelo y devuelve la vista adecuada: la confirmación si todo salió bien, o el formulario con los errores si el modelo lanzó una validación. La vista no decide reglas y el modelo no conoce nada de formularios.

Beneficio: el modelo se prueba sin interfaz y la interfaz se reemplaza sin tocar el modelo. Nota: si se usa Django, su equivalente se llama MTV (Model, Template, View), donde views.py hace de controlador.

Patrón	Donde se aplica	Problema de refugio que resuelven	Beneficio técnico
Singleton	ConexionBD, MotorCasos	Conexiones y buses de eventos duplicados	Instancia única y segura para hilos
Factory Method	FabricaExpediente y subclases	Reglas distintas por especie	Nuevas especies sin tocar el motor
Observer	SujetoRescates y observadores	Avisar a voluntarios sin acoplar el motor	Reacciones nuevas sin modificar lo existente
MVC	modelo, vista, controlador	Lógica mezclada con la interfaz	Pruebas aisladas e interfaz intercambiable

Complementariamente, el diseño usa State para los estados del caso, Strategy para la asignación de responsables y Repository para el acceso a datos, que son compatibles con esta estructura.

Beneficios del sistema

- Mejora el control de los animales rescatados.
- Facilita la búsqueda de animales disponibles para adopción.
- Ayuda a mantener actualizados los historiales médicos.
- Reduce errores y pérdida de información.
- Promueve la adopción responsable.
- Facilita la elaboración de estadísticas para conocer el impacto de la organización.

Diagramas

Los diagramas están escritos en Mermaid: GitHub los dibuja automáticamente al pegarlos en un archivo .md del repositorio (por ejemplo docs/clases-patrones.md).

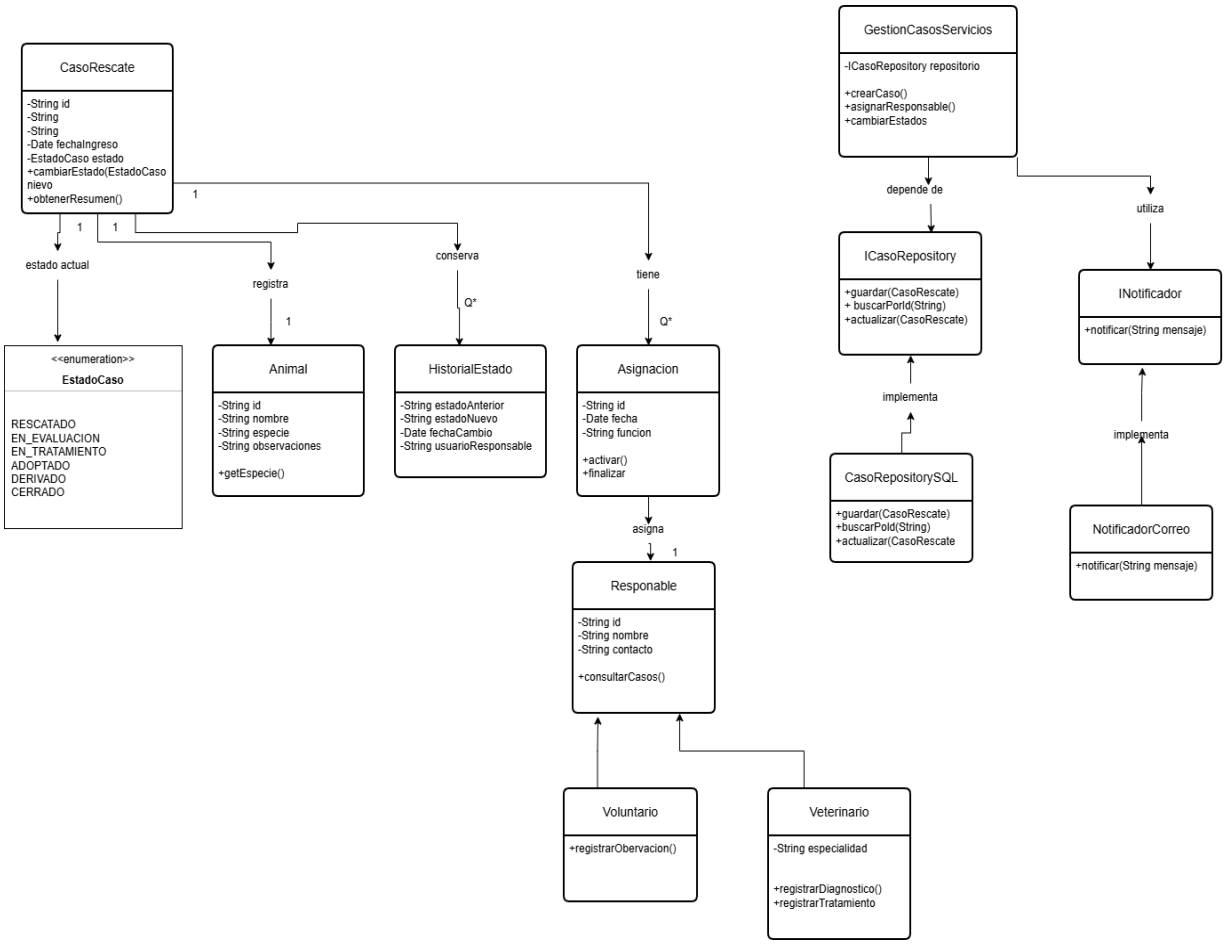
Explicación del diagrama

- Ciudadano: registra el reporte inicial y proporciona información sobre el animal. La consulta del estado debe estar limitada a los casos que pueda consultar.
- Voluntario o veterinario: consulta sus asignaciones y actualiza información de acuerdo con sus responsabilidades. Los diagnósticos y tratamientos corresponden al veterinario autorizado.
- Administrador: coordina las asignaciones, administra usuarios y supervisa los reportes y el historial de casos.

El sistema debe validar los datos recibidos y los permisos antes de ejecutar cualquier operación.

Diagrama de clases UML aplicando principios SOLID

El diagrama de clases representa las entidades, atributos, operaciones e interfaces que componen el sistema. La propuesta utiliza clases de dominio, servicios y repositorios para separar las responsabilidades.



Justificación del uso de Gitflow

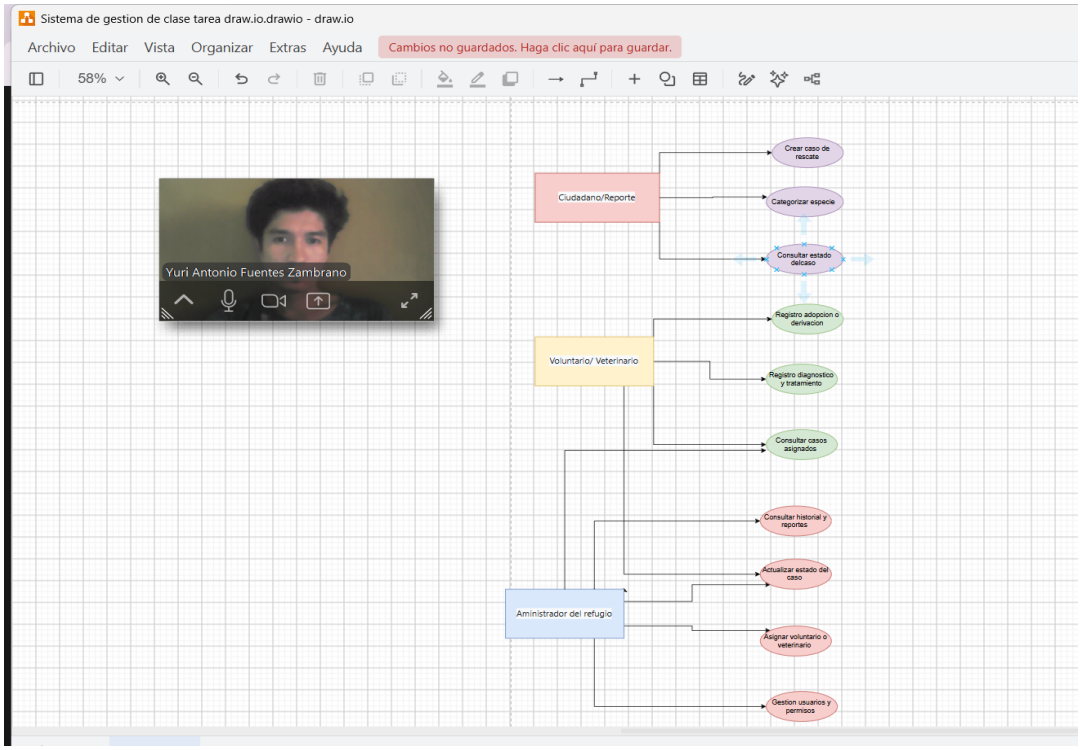
Gitflow permite organizar el trabajo en ramas con objetivos definidos. Para este proyecto se utilizará el repositorio público de GitHub como punto central de colaboración y seguimiento de los cambios.

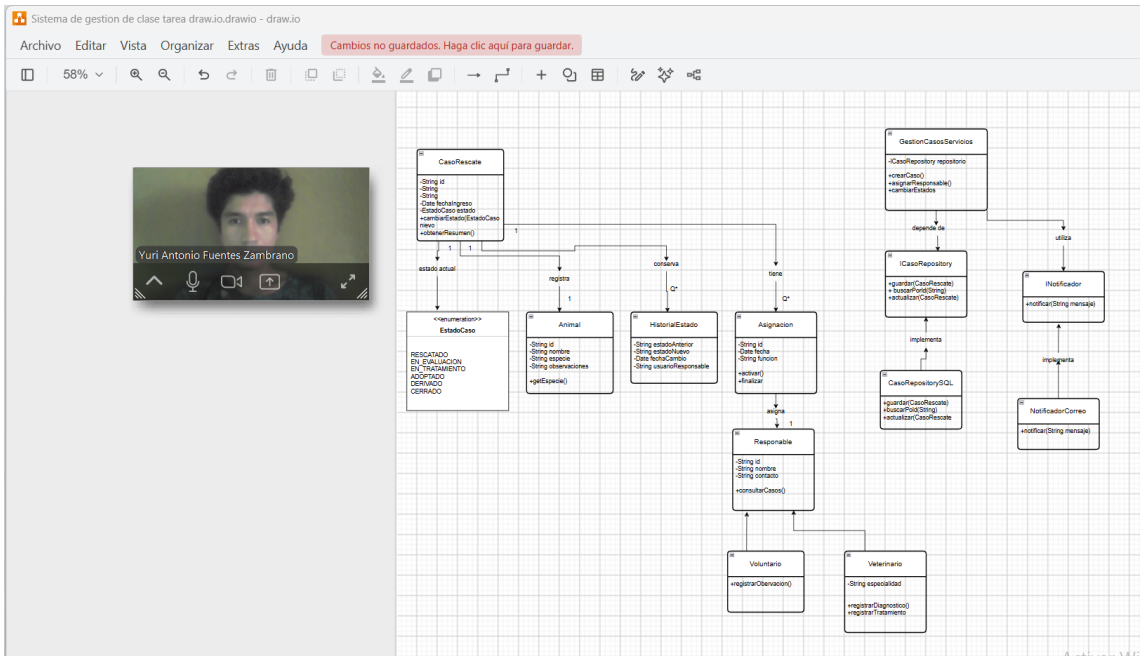
Gitflow se eligió porque este proyecto tiene características que encajan con su modelo:

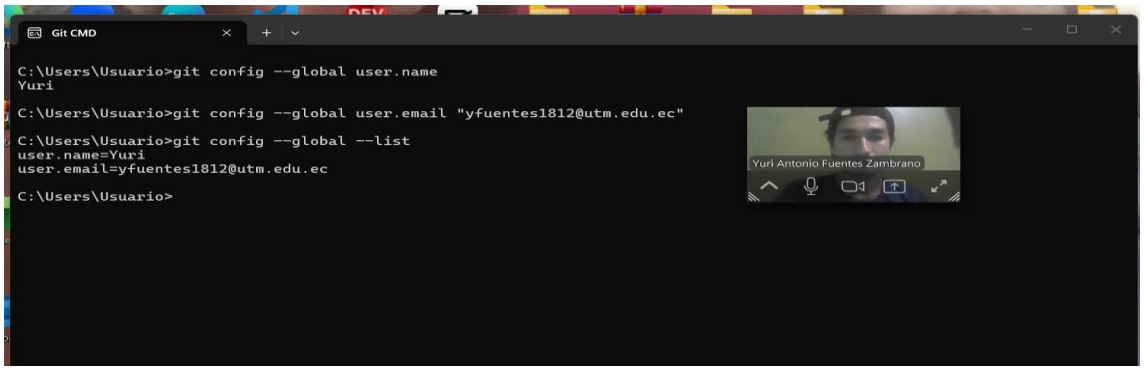
- Hay varios módulos que avanzan en paralelo (patrones, modelo, controlador, documentación). Una rama feature por tarea evita que el trabajo a medias de una persona bloquee a otra.
- El sistema manejará datos personales de adoptantes. La rama master solo recibe código estable y etiquetado, así que producción nunca depende de un cambio sin revisar.
- Las versiones se publican por etapas. La rama release permite congelar una versión, corregir detalles finales y etiquetarla (v1.0.0) sin detener el desarrollo en develop.
- Los errores en producción se corrigen sin esperar al siguiente ciclo. Una rama hotfix sale de master, se corrige y se fusiona de vuelta en master y develop.

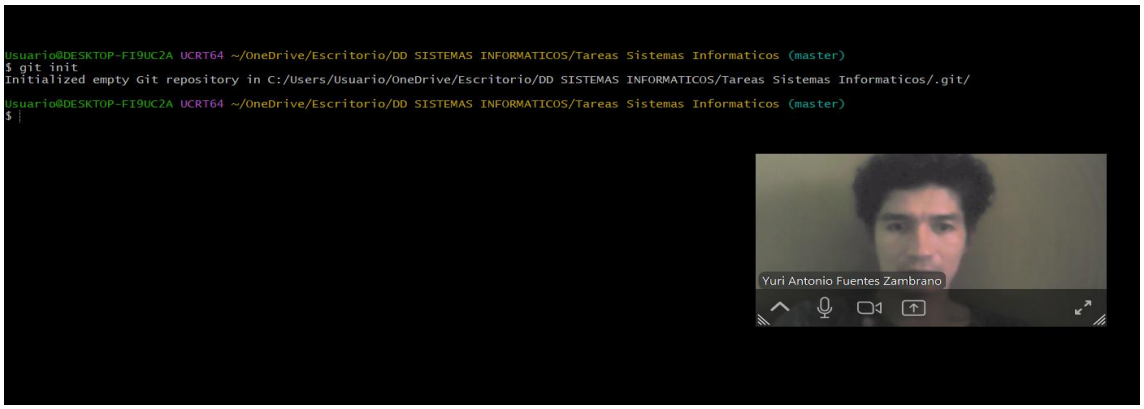
- Los voluntarios tienen experiencia técnica distinta. Las reglas claras (nombres de ramas, Pull Request obligatorio, revisión) funcionan como guía y como control de calidad.
  Limitación reconocida: Gitflow añade más ceremonia que un flujo basado en una sola rama principal. Se aceptó porque el proyecto tiene entregas versionadas y porque el objetivo académico incluye demostrar el flujo completo.

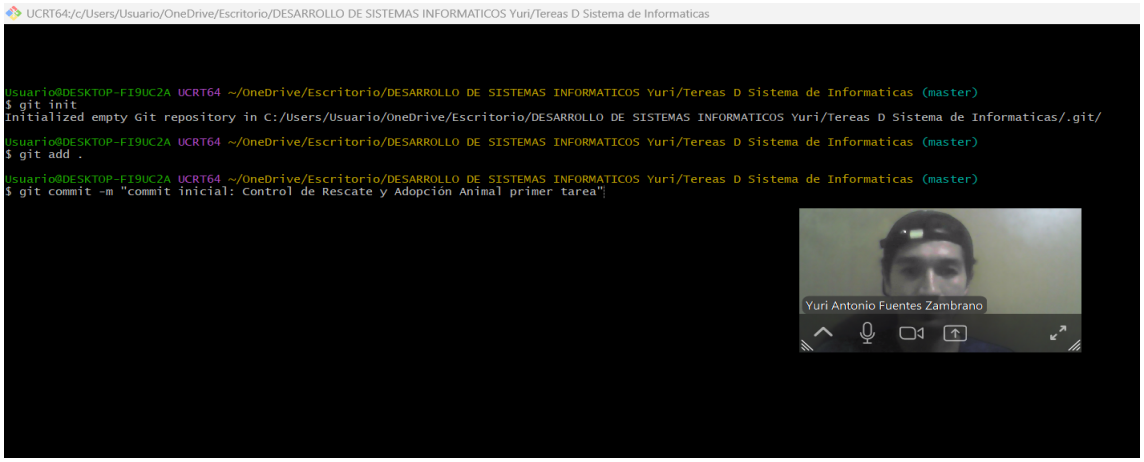
Evidencias

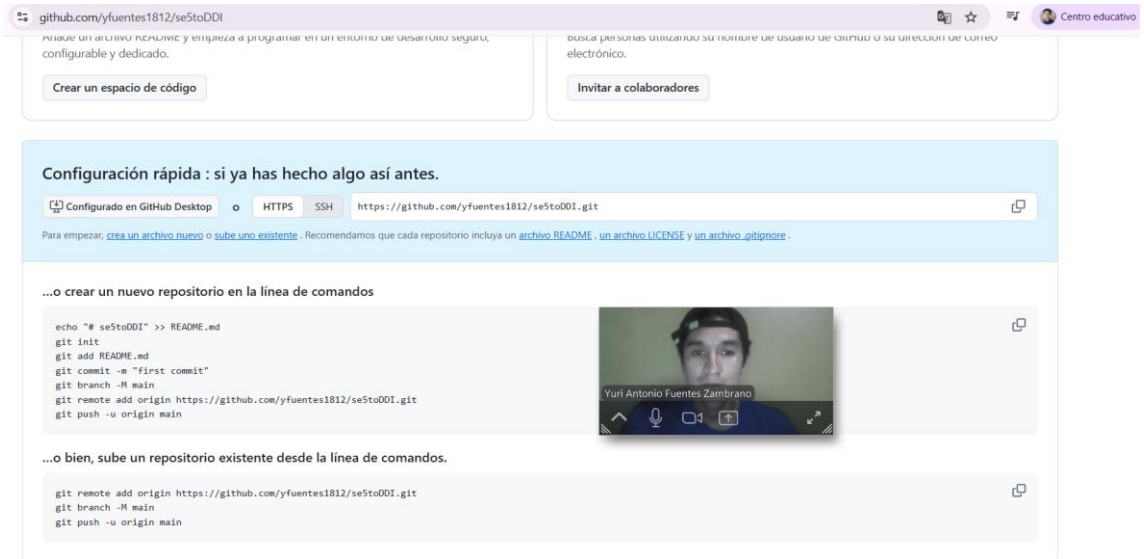





Git CMD
C:\Users\Usuario>git config --global user.name
Yuri
C:\Users\Usuario>git config --global user.email "yfuentes1812@utm.edu.ec"
C:\Users\Usuario>git config --global --list
user.name=Yuri
user.email=yfuentes1812@utm.edu.ec
C:\Users\Usuario>
Yuri Antonio Fuentes Zambrano


Usuario@DESKTOP-FI9UC2A UCRT64 ~/OneDrive/Escritorio/DD SISTEMAS INFORMATICOS/Tareas Sistemas Informaticos (master)
$ git init
Initialized empty Git repository in C:/Users/Usuario/OneDrive/Escritorio/DD SISTEMAS INFORMATICOS/Tareas Sistemas Informaticos/.git/
Usuario@DESKTOP-FI9UC2A UCRT64 ~/OneDrive/Escritorio/DD SISTEMAS INFORMATICOS/Tareas Sistemas Informaticos (master)
$
Yuri Antonio Fuentes Zambrano


UCRT64:/c/Users/Usuario/OneDrive/Escritorio/DESARROLLO DE SISTEMAS INFORMATICOS Yuri/Tereas D Sistema de Informaticas
Usuario@DESKTOP-FI9UC2A UCRT64 ~/OneDrive/Escritorio/DESARROLLO DE SISTEMAS INFORMATICOS Yuri/Tereas D Sistema de Informaticas (master)
$ git init
Initialized empty Git repository in C:/Users/Usuario/OneDrive/Escritorio/DESARROLLO DE SISTEMAS INFORMATICOS Yuri/Tereas D Sistema de Informaticas/.git/
Usuario@DESKTOP-FI9UC2A UCRT64 ~/OneDrive/Escritorio/DESARROLLO DE SISTEMAS INFORMATICOS Yuri/Tereas D Sistema de Informaticas (master)
$ git add .
Usuario@DESKTOP-FI9UC2A UCRT64 ~/OneDrive/Escritorio/DESARROLLO DE SISTEMAS INFORMATICOS Yuri/Tereas D Sistema de Informaticas (master)
$ git commit -m "commit inicial: Control de Rescate y Adopción Animal primer tarea"
Yuri Antonio Fuentes Zambrano


github.com/yfuentes1812/se5toDDI
Centro educativo
configurable y dedicado.
Crear un espacio de código
electrónico.
Invitar a colaboradores
Configuración rápida : si ya has hecho algo así antes.
Configurado en GitHub Desktop
HTTPS
SSH
https://github.com/yfuentes1812/se5toDDI.git
Para empezar, crea un archivo nuevo o sube uno existente . Recomendamos que cada repositorio incluya un archivo README , un archivo LICENSE y un archivo .gitignore .
...o crear un nuevo repositorio en la línea de comandos
echo "# se5toDDI" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/yfuentes1812/se5toDDI.git
git push -u origin main
Yuri Antonio Fuentes Zambrano
...o bien, sube un repositorio existente desde la línea de comandos.
git remote add origin https://github.com/yfuentes1812/se5toDDI.git
git branch -M main
git push -u origin main

Bibliografía

Carrión, J. &. (2021). Impacto de las plataformas digitales en los índices de adopción de animales de compañía en abandono. *Revista de Ciencias Veterinarias y Bienestar Animal, 14*(12).

Gallegos Ortíz, G. S. (2018). Evaluación de bienestar animal mediante un sistema de control de calidad en el refugio Paraíso Huellas Rescate Animal en la parroquia Guayllabamba del cantón Quito. *Trabajo de Titulacion*. https://doi.org/http://dspace.udla.edu.ec/handle/33000/8887

Pilatuña Aguagallo, M. D. (2023). Implementación de un sistema RFID para el control de la salud y bienestar en el centro de rescate animal CRIAR-Riobamba. *(Bachelor's thesis, Riobamba, Universidad Nacional de Chimborazo).*